



- [Downloads](#)
- [Documentation](#)
- [Get Involved](#)
- [Help](#)



.

Q

Search docs

Q

≡

[PHP 8.4.24 Released!](#)

[Getting Started](#)

[Introduction](#)

[A simple tutorial](#)

[Language Reference](#)

[Basic syntax](#)

[Types](#)

[Variables](#)

[Constants](#)

[Expressions](#)

[Operators](#)

[Control Structures](#)

[Functions](#)

[Classes and Objects](#)

[Namespaces](#)

[Enumerations](#)

[Errors](#)

[Exceptions](#)

[Fibers](#)

[Generators](#)

[Attributes](#)

[References Explained](#)

[Predefined Variables](#)

[Predefined Exceptions](#)

[Predefined Interfaces and Classes](#)

[Predefined Attributes](#)

[Context options and parameters](#)

[Supported Protocols and Wrappers](#)

[Security](#)

[Introduction](#)

[General considerations](#)

[Installed as CGI binary](#)

[Installed as an Apache module](#)

[Session Security](#)

[Filesystem Security](#)

[Database Security](#)

[Error Reporting](#)

[User Submitted Data](#)

[Hiding PHP](#)

[Keeping Current](#)

[Features](#)

[HTTP authentication with PHP](#)

[Cookies](#)

[Sessions](#)

[Handling file uploads](#)

[Using remote files](#)

[Connection handling](#)

[Persistent Database Connections](#)

[Command line usage](#)

[Garbage Collection](#)

[DTrace Dynamic Tracing](#)

[Function Reference](#)

[Affecting PHP's Behaviour](#)

[Audio Formats Manipulation](#)

[Authentication Services](#)

[Command Line Specific Extensions](#)
[Compression and Archive Extensions](#)
[Cryptography Extensions](#)
[Database Extensions](#)
[Date and Time Related Extensions](#)
[File System Related Extensions](#)
[Human Language and Character Encoding Support](#)
[Image Processing and Generation](#)
[Mail Related Extensions](#)
[Mathematical Extensions](#)
[Non-Text MIME Output](#)
[Process Control Extensions](#)
[Other Basic Extensions](#)
[Other Services](#)
[Search Engine Extensions](#)
[Server Specific Extensions](#)
[Session Extensions](#)
[Text Processing](#)
[Variable and Type Related Extensions](#)
[Web Services](#)
[Windows Only Extensions](#)
[XML Manipulation](#)
[GUI Extensions](#)

Keyboard Shortcuts

? This help
j Next menu item
k Previous menu item
g p Previous man page
g n Next man page
G Scroll to bottom
g g Scroll to top
g h Goto homepage
g s Goto search
(current page)
/ Focus search box

[Error Reporting »](#)
[« Encrypted Storage Model](#)

- [Preface](#)
- [Security](#)
- [Database Security](#)

Change language:

SQL Injection ¶

SQL injection is a technique where an attacker exploits flaws in application code responsible for building dynamic SQL queries. The attacker can gain access to privileged sections of the application, retrieve all information from the database, tamper with existing data, or even execute dangerous system-level commands on the database host. The vulnerability occurs when developers concatenate or interpolate arbitrary input in their SQL statements.

Example #1 Splitting the result set into pages ... and making superusers (PostgreSQL)

In the following example, user input is directly interpolated into the SQL query allowing the attacker to gain a superuser account in the database.

```
<?php
$offset = $_GET['offset']; // beware, no input validation!
$query = "SELECT id, name FROM products ORDER BY name LIMIT 20 OFFSET $offset;";
$result = pg_query($conn, $query);

?>
```

Normal users click on the 'next', 'prev' links where the *\$offset* is encoded into the URL. The script expects that the incoming *\$offset* is a number. However, what if someone tries to break in by appending the following to the URL

```
0;
insert into pg_shadow(username,usesysid,usesuper,usecatupd,passwd)
select 'crack', usesysid, 't','t','crack'
```

```
-- from pg_shadow where username='postgres';
```

If it happened, the script would present a superuser access to the attacker. Note that `0;` is to supply a valid offset to the original query and to terminate it.

Note:

It is a common technique to force the SQL parser to ignore the rest of the query written by the developer with `--` which is the comment sign in SQL.

A feasible way to gain passwords is to circumvent your search result pages. The only thing the attacker needs to do is to see if there are any submitted variables used in SQL statements which are not handled properly. These filters can be set commonly in a preceding form to customize `WHERE`, `ORDER BY`, `LIMIT` and `OFFSET` clauses in `SELECT` statements. If your database supports the `UNION` construct, the attacker may try to append an entire query to the original one to list passwords from an arbitrary table. It is strongly recommended to store only secure hashes of passwords instead of the passwords themselves.

Example #2 Listing out articles ... and some passwords (any database server)

```
<?php
$query = "SELECT id, name, inserted, size FROM products
        WHERE size = '$size'";
$result = odbc_exec($conn, $query);

?>
```

The static part of the query can be combined with another `SELECT` statement which reveals all passwords:

```
,
union select '1', concat(uname||'-'||passwd) as name, '1971-01-01', '0' from usertable;
--
```

`UPDATE` and `INSERT` statements are also susceptible to such attacks.

Example #3 From resetting a password ... to gaining more privileges (any database server)

```
<?php
$query = "UPDATE usertable SET pwd='$pwd' WHERE uid='$uid'";
?>
```

If a malicious user submits the value `'` or uid like '%admin%' to `$uid` to change the admin's password, or simply sets `$pwd` to `hehehe`, `trusted=100`, `admin='yes` to gain more privileges, then the query will be twisted:

```
<?php
// $uid: ' or uid like '%admin%'
$query = "UPDATE usertable SET pwd='...' WHERE uid='' or uid like '%admin%'";
// $pwd: hehehe', trusted=100, admin=yes
$query = "UPDATE usertable SET pwd='hehehe', trusted=100, admin='yes' WHERE
...;";

?>
```

While it remains obvious that an attacker must possess at least some knowledge of the database architecture to conduct a successful attack, obtaining this information is often very simple. For example, the code may be part of an open-source software and be publicly available. This information may also be divulged by closed-source code - even if it's encoded, obfuscated, or compiled - and even by your own code through the display of error messages. Other methods include the use of typical table and column names. For example, a login form that uses a 'users' table with column names 'id', 'username', and 'password'.

Example #4 Attacking the database host operating system (MSSQL Server)

A frightening example of how operating system-level commands can be accessed on some database hosts.

```
<?php
$query = "SELECT * FROM products WHERE id LIKE '%$prod%'";
$result = mssql_query($query);

?>
```

If attacker submits the value `a%` `exec master..xp_cmdshell 'net user test testpass /ADD' --` to `$prod`, then the `$query` will be:

```
<?php
$query = "SELECT * FROM products
        WHERE id LIKE '%a%'
        exec master..xp_cmdshell 'net user test testpass /ADD' --%";
$result = mssql_query($query);

?>
```

MSSQL Server executes the SQL statements in the batch including a command to add a new user to the local accounts database. If this application were running as `sa` and the `MSSQLSERVER` service was running with sufficient privileges, the attacker would now have an account with which to access this machine.

Note: The `mssql_query()` function belonged to the `mssql` extension, which was [removed in PHP 7.0.0](#). It is used here only to illustrate the attack; current code connecting to MSSQL Server should use the [sqlsrv](#) or [pdo_sqlsrv](#) extensions instead.

Note:

Some examples above are tied to a specific database server, but it does not mean that a similar attack is impossible against other products. Your database server may be similarly vulnerable in another manner.

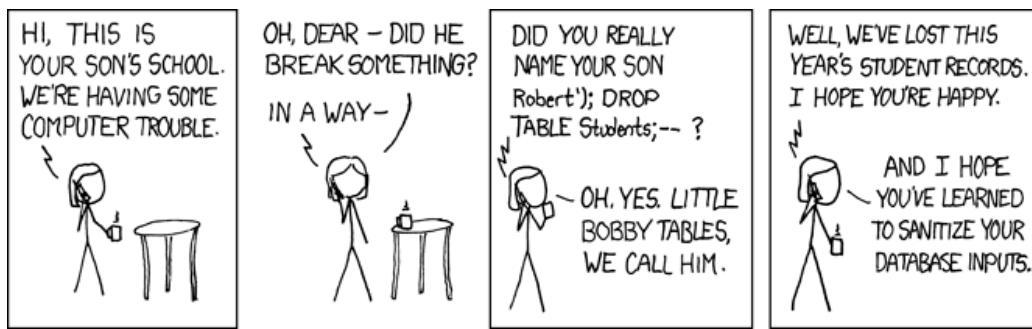


Image courtesy of » [xkcd](#)

Avoidance Techniques ¶

The recommended way to avoid SQL injection is by binding all data via prepared statements. Using parameterized queries isn't enough to entirely avoid SQL injection, but it is the easiest and safest way to provide input to SQL statements. All dynamic data literals in WHERE, SET, and VALUES clauses must be replaced with placeholders. The actual data will be bound during the execution and sent separately from the SQL command.

Parameter binding can only be used for data. Other dynamic parts of the SQL query must be filtered against a known list of allowed values.

Example #5 Avoiding SQL injection by using PDO prepared statements

```
<?php
// The dynamic SQL part is validated against expected values
$sortingOrder = $_GET['sortingOrder'] === 'DESC' ? 'DESC' : 'ASC';
$productId = $_GET['productId'];
// The SQL is prepared with a placeholder
$stmt = $pdo->prepare("SELECT * FROM products WHERE id LIKE ? ORDER BY price {$sortingOrder}");
// The value is provided with LIKE wildcards
$stmt->execute(["%{$productId}%"]);
?>
```

Prepared statements are provided [by PDO](#), [by MySQLi](#), and by other database libraries.

SQL injection attacks are mainly based on exploiting the code not being written with security in mind. Never trust any input, especially from the client side, even though it comes from a select box, a hidden input field, or a cookie. The first example shows that such a simple query can cause disasters.

A defense-in-depth strategy involves several good coding practices:

- Never connect to the database as a superuser or as the database owner. Use always customized users with minimal privileges.
- Check if the given input has the expected data type. PHP has a wide range of input validating functions, from the simplest ones found in [Variable Functions](#) and in [Character Type Functions](#) (e.g. [is_numeric\(\)](#), [ctype_digit\(\)](#) respectively) and onwards to the [Perl Compatible Regular Expressions](#) support.
- If the application expects numerical input, consider verifying data with [ctype_digit\(\)](#), silently change its type using [settype\(\)](#), or use its numeric representation by [sprintf\(\)](#).
- If the database layer doesn't support binding variables then quote each non-numeric user-supplied value that is passed to the database with the database-specific string escape function (e.g. [mysql_real_escape_string\(\)](#), [sqlite_escape_string\(\)](#), etc.). Generic functions like [addslashes\(\)](#) are useful only in a very specific environment (e.g. MySQL in a single-byte character set with disabled [NO_BACKSLASH_ESCAPES](#)), so it is better to avoid them.
- Do not print out any database-specific information, especially about the schema, by fair means or foul. See also [Error Reporting](#) and [Error Handling and Logging Functions](#).

Besides these, you benefit from logging queries either within your script or by the database itself, if it supports logging. Obviously, the logging is unable to prevent any harmful attempt, but it can be helpful to trace back which application has been circumvented. The log is not useful by itself but through the information it contains. More detail is generally better than less.

Found A Problem?

[Learn How To Improve This Page](#) • [Submit a Pull Request](#) • [Report a Bug](#)

[+ add a note](#)

User Contributed Notes 1 note

[up](#)
[down](#)

41
[Richard dot Corfield at gmail dot com ¶](#)
14 years ago

The best way has got to be parameterised queries. Then it doesn't matter what the user types in the data goes to the database as a value.

A quick search online shows some possibilities in PHP which is great! Even on this site - <http://php.net/manual/en/pdo.prepared-statements.php> which also gives the reasons this is good both for security and performance.

[+ add a note](#)

- [Database Security](#)
 - [Designing Databases](#)
 - [Connecting to Database](#)
 - [Encrypted Storage Model](#)
 - [SQL Injection](#)
- [Copyright © 2001-2026 The PHP Documentation Group](#)
- [My PHP.net](#)
- [Contact](#)
- [Other PHP.net sites](#)
- [Privacy policy](#)



Search docs



↑ and ↓ to navigate • Enter to select • Esc to close • / to open
Press Enter without selection to search using Google